Certificate-grounded evaluation: when the answer key is a proof

graders graded, answer keys re-decided, a time-horizon fit certified, and a pre-registered benchmark

Carlos Toledo*

October 2026 — draft v0.1 for review; every number interpolated from the record; machine-derived, not peer-reviewed; author list to be agreed

Abstract

The false-accept rate of a verifier is measured today against labels, behavioural diffs and human adjudicators, none of which is a proof, and the field's standard remedy for numeric grading is to tighten the tolerance. We study the corner of the problem where a certificate exists: an interval $[\mathrm{lo}, \mathrm{hi}]$ proved to contain the quantity by outward-rounded interval or exact rational arithmetic. There the set of accepted-but-wrong values of a tolerance grader has a closed form, $2\,\mathrm{tol} - w$ for a certificate of width w; it is empty only when the grader has stopped being a tolerance grader; and it is a generator, since provably wrong submissions can be minted from certificates already on disk rather than invented. Four results are built on that observation, each from a public record re-run at every build. (i) A canary factory of 104 facts and 4,418 submissions grades four reference graders: absolute tolerance accepts 89.5% of 4,000 provably wrong values, and still 44.4% of them at a tolerance of 1×10^{-15}; the certificate grader accepts none and rejects none of the 418 provably right ones. One canary was not minted: a constant a problem thread published, 6.09×10^{-13} outside its certificate. (ii) Three specimens of answer keys failing in ways a rerun cannot catch, and one published claim the same instruments confirm. (iii) GSM8K's answer key re-decided to the last step: all 4,282 test-set calculator annotations are exact, 2 test and 24 train keys print a step that does not hold, and both test slips sit where no model disagreed and so no human looked. (iv) METR's Time Horizon 1.1 fits certified: for 23 models the penalised logistic fit is proved to hold exactly one optimum in a box of radius below 10^{-10}, the printed coefficients are its rounding for 22 of 23, and the post-2023 doubling time re-derives as 128.740 days against the printed 128.744. A verified reward channel and three public environments turn the same grammar into training signal, and a pre-registered benchmark of 46 constructions decided exactly measures three models under it. The novelty claimed is narrow: the decision grammar—certified, refuted, refused, with controls that must fire—and the public, re-runnable record behind every number here.

1 Introduction

When a model is trained or evaluated against a machine-checkable task, the reward is a program: it parses the output, extracts an answer, compares a value, runs a test and returns a number.

*Independent researcher. Contact: carlos@carlostoledo.co. This work was produced with substantial AI assistance in a certified-verification setting: constructions and exposition were proposed with the help of large language models, and every quantitative claim is decided by outward-rounded interval arithmetic or exact rational arithmetic, with adversarial controls that must fire for the certificate to be accepted. The certificates, not the models, are the evidence, and they are public and re-runnable. Machine-derived and not peer-reviewed.

Bugs in that program do not add noise; under optimisation pressure they become the objective. The size of the problem is not in dispute and none of the following numbers are ours. Of a 49-task sample of SWE-bench Verified, 28.5% have test suites weak enough to accept an incorrect patch, as do 25.0% of a 20-task R2E-Gym sample, and across 134 frontier-model submissions the tasks flagged hackable inflate within-stratum Pass@1 by 14.14 percentage points [1]. OpenAI reported in February 2026 that 59.4% of failed tasks on the same benchmark have flawed tests, the benchmark's release audit having been a manual sweep by 93 developers [2]. A preregistered causal contrast on MBPP puts the leak-stratum false-positive share 43.8 points above clean tasks and finds that 47.57% of rewarded false positives are, under signed human adjudication, wrong code [3]. Two otherwise identical 7B models differing only in verifier design show a 35% reward gap [4]. Deliberately buggy mathematics, tool-call and code verifiers accept incorrect completions at rates 0.832, 0.869 and 0.557, and a replay against the open-source `math-verify` gives 10 of 60 [5].

This paper is about one corner of that surface: the case where the claimed answer is a number, or an exhibited object, and where a *certificate* for it can exist. In that corner the question "is this submission wrong?" has an answer that depends on nobody's key. The thesis has three parts. Where a certificate exists the negative set of a grader can be proved rather than labelled, and generated in closed form rather than mutated (§3–4). The standard remedy, tightening the tolerance, provably does not close the hole: it shrinks the band linearly and closes it only where the grader has become a certificate grader with gratuitous rejections (Proposition 1). And the grammar that grades a number also grades an answer key, a published fit and a model's construction, every such grading being a public record that re-runs (§5–9).

What is new is deliberately limited. Verifier soundness is an active 2026 literature [1, 3, 4, 5] and environments with designer-embedded reward hacks are already published; interval arithmetic with outward rounding, the Krawczyk operator and exact rational certification are classical [11, 12, 13]. The contribution is the decision grammar—CERTIFIED, REFUTED, REFUSED; a screen that may only prune; red controls that must fire before anything real is graded—applied uniformly to graders, answer keys, a published estimator and a benchmark, together with the record: every number in this paper is a LaTeX macro written by a program from the certificate ledgers of the `cert-machine` repository [25], and that program refuses to write when a record no longer supports a sentence.

2 What "wrong" means, and why it is usually a comparison

Three recent studies measure how often a verifier accepts an incorrect submission. They agree that the rate is large, they disagree about almost everything else, and they establish incorrectness in three different ways, none of which is a proof, as each says of itself. Ray [5] grades against a label supplied by the workload design, with a stricter verifier as an operational proxy, and states the limitation plainly: "$T(c)$ is the intended semantic label supplied by the workload design. The strict verifier is an operational proxy for $T(c)$, not a formal proof of correctness. Accordingly, a strict false-positive rate of zero means zero on the generated cases in this controlled evaluation, not zero for all possible completions or deployments." Rajan [1] establishes wrongness by an LLM-generated patch that changes observable behaviour and still passes; Zhang [3] by signed, human-adjudicated rules. When wrongness is a label, a measured false-accept rate is a disagreement between two graders and can be attacked by attacking the reference; when it is a behavioural diff, the rate inherits whatever the diff missed; when it is an adjudication, the

rate inherits the adjudicator. In each case the number is an estimate of a quantity nobody can exhibit.

Definition 1 (certificate)**.** Let q be a real quantity. A *certificate* for q is a pair of exact rationals $[\mathrm{lo}, \mathrm{hi}]$ together with a machine-checkable derivation that $\mathrm{lo} \le q \le \mathrm{hi}$, produced by outward-rounded interval arithmetic or exact rational arithmetic with no floating-point operation in any decision. Its *width* is $w = \mathrm{hi} - \mathrm{lo}$. A submitted value v is *refuted* by the certificate when $v < \mathrm{lo}$ or $v > \mathrm{hi}$. Throughout, *certified* means exactly this and nothing weaker: such a derivation exists and has been re-run. A claim the instrument cannot decide is *refused*, never guessed.

A refutation in this sense is not a comparison against a reference answer; it holds against every possible reference at once. If a grader accepts a value that a certificate refutes, the false accept is a theorem about the grader, and the rate stops being an estimate. Fix a stored reference $k \in [\mathrm{lo}, \mathrm{hi}]$ (a correct decimal, as an answer key is supposed to be) and a tolerance $\mathrm{tol} > 0$. The grader shapes measured below are the ones deployed in practice: *absolute tolerance*, accept v iff $|v - k| < \mathrm{tol}$; *relative tolerance*, accept iff $|v - k|/|k| < \mathrm{tol}$; *exact match*, accept iff $v = k$ as a literal; and *certificate-grounded*, accept iff $\mathrm{lo} \le v \le \mathrm{hi}$. Only the last consults the proof.

3 The acceptance band

Write $A = (k - \mathrm{tol},\, k + \mathrm{tol})$ for the values an absolute-tolerance grader accepts and $R^c = (-\infty, \mathrm{lo}) \cup (\mathrm{hi}, \infty)$ for the values the certificate refutes. The *acceptance band* is $B = A \cap R^c$, the values simultaneously accepted and proved wrong.

Proposition 1 (the band)**.** *If k is the midpoint of $[\mathrm{lo}, \mathrm{hi}]$ and $\mathrm{tol} > w/2$ then $|B| = 2\,\mathrm{tol} - w$, and B is empty precisely when $\mathrm{tol} \le w/2$.*

Proof. $B = (k - \mathrm{tol}, \mathrm{lo}) \cup (\mathrm{hi}, k + \mathrm{tol})$; with $k = (\mathrm{lo} + \mathrm{hi})/2$ each part has length $\mathrm{tol} - w/2$, positive exactly when $\mathrm{tol} > w/2$, and the two parts are disjoint. □

Define the *band ratio* $\rho = |B|/w = 2\,\mathrm{tol}/w - 1$, the number of certificate widths of wrongness the grader accepts; it depends on the ratio of tolerance to width and on neither alone. The bug taxonomy of [5] lists the failure directly ("loose tolerance: accepts nearby wrong value") and prescribes a tight numeric tolerance. Proposition 1 says the fix is insufficient in a quantitative way: tightening tol shrinks $|B|$ linearly and closes it only at $\mathrm{tol} = w/2$, where the grader accepts a sub-interval of the certificate and has become a certificate grader with gratuitous false rejects. For the widths this laboratory holds the band is large: the median non-degenerate certificate in the corpus of §4 has width 3.6×10^{-15}, so a routine $\mathrm{tol} = 10^{-9}$ gives $\rho \approx 562{,}949$; the narrowest, $\sigma^* = 1/(8\pi^2)$ (`terra.sigmaStar`), has width 1.56×10^{-17} and $\rho \approx 128{,}102{,}388$.

The band is computed in the reals; a model submits a double. Those are different questions, and the gap is itself a rung.

Proposition 2 (the band in floating point)**.** *Let u be the spacing of IEEE-754 doubles at the magnitude of* hi*. If $w = 0$ (an exact integer or rational certificate) and $\mathrm{tol} < u$, then B is a non-empty interval of reals containing no representable double, and every double a submitter can produce is either equal to the certified value or rejected by the grader.*

Proof. $B = (k - \mathrm{tol}, k) \cup (k, k + \mathrm{tol})$ has positive length; the doubles nearest to k other than k are at distance at least $u > \mathrm{tol}$, so none lies in B; a double in A must then equal k. □

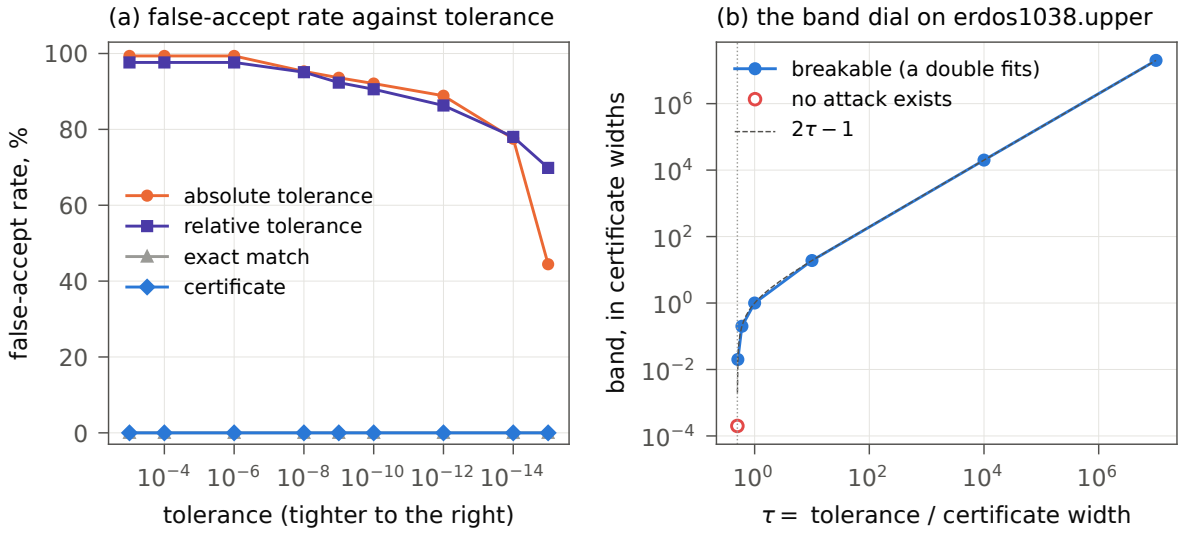


Figure 1: (a) False-accept rate of the four reference graders against the tolerance, over the 4,000 provably wrong submissions of §4; the exact-match and certificate rows lie on the axis. (b) The band of accepted-but-wrong values in certificate widths against $\tau = \text{tol}/w$ on `erdos1038.upper` (width 3.4×10^{-13}): the closed form $2\tau - 1$, and the point past which no double fits. Drawn from `certs/envs-record.json` and `certs/gym-record.json`.

Around the integer 64 with $\text{tol} = 10^{-15}$ the band is a perfectly good interval of reals and the nearest double is 1.4×10^{-14} away; a submitter reasoning "$64 + 5 \times 10^{-16}$ will do" submits a value that *is* 64 in float64, inside the certificate, and is scored wrong. The rung exists because the laboratory's own generator had the identical bug, $\text{hi} + \text{tol}/2$ rounding back to hi on a zero-width certificate, and its battery caught it before it shipped. The environment of §8 therefore measures difficulty in *room*, the number of representable doubles that fit in the band, and solves $\text{tol} = (w + \text{room} \cdot u)/2$ for the tolerance it wants. Figure 1(b) draws the dial on one fact: the band in certificate widths falls as $2\tau - 1$ in $\tau = \text{tol}/w$ and stops being reachable before it vanishes.

4 The canary factory: graders graded

The band is a generator: every value in B is a submission that is provably wrong and guaranteed to pass, so the adversarial set is enumerated rather than mutated, and it is unbounded. The suite is built from 104 certified facts, each read from a record in the repository at load time and pinned to it by sha256 (103 of them; the last is a closed form enclosed from an enclosure of π at build): the two constants of Erdős problem #852, the Erdős–Herzog–Piranian bracket, Chowla's cosine dips, and the conjectures of the laboratory's own ledger, 52 of them exact integers of width zero (a tensor rank, a contact count, a period count). Minting from a fact not marked certified throws rather than degrading, and a record whose bytes have moved makes the battery refuse. Tolerances probed: 1×10^{-3}, 1×10^{-4}, 1×10^{-6}, 1×10^{-8}, 1×10^{-9}, 1×10^{-10}, 1×10^{-12}, 1×10^{-14}, 1×10^{-15}. Submissions: 4,418, of which 4,000 are provably wrong and 418 provably right, the second half because a false-accept benchmark with no controls is passed perfectly by a grader that rejects everything. Table 1 lists the families and Table 2 the measurement; soundness is

$(1 - \mathrm{FA})(1 - \mathrm{FR})$ because either rate alone is gameable.

family	submissions	truth
tolerance-interior: inside the window, outside the certificate	1,649	wrong
ulp-outside: the first doubles past each endpoint	1,872	wrong
over-truncated: the value printed to too few digits	468	wrong
control: provably right, drawn from inside the certificate	418	right
published-wrong: a value a problem thread printed	9	wrong
float-cancellation: a naive sum whose true value is 0	2	wrong

Table 1: The canary families of `certs/envs-record.json`. The published-wrong family carries, at every tolerance, the value a problem thread printed for `erdos852.cstar`.

grader	false-accept	false-reject	soundness
absolute tolerance, $\lvert v - k\rvert < \mathrm{tol}$	89.5%	0.5%	10.4%
relative tolerance, $\lvert v - k\rvert/\lvert k\rvert < \mathrm{tol}$	90.3%	0.5%	9.6%
exact match, $v = k$ as a literal	0.0%	24.4%	75.6%
certificate, $\mathrm{lo} \le v \le \mathrm{hi}$	0.0%	0.0%	100.0%

tolerance	absolute	relative
1×10^{-3}	99.4%	97.6%
1×10^{-4}	99.4%	97.6%
1×10^{-6}	99.4%	97.6%
1×10^{-8}	95.3%	95.1%
1×10^{-9}	93.6%	92.3%
1×10^{-10}	92.1%	90.6%
1×10^{-12}	88.9%	86.3%
1×10^{-14}	77.5%	78.0%
1×10^{-15}	44.4%	69.8%

Table 2: Left: the four graders over the whole suite. Right: the false-accept rate of the two tolerance graders by tolerance; the exact-match and certificate graders are zero at every tolerance.

Three readings. The absolute-tolerance grader accepts 89.5% of the provably wrong submissions and the relative one 90.3%; against the 418 controls each rejects 0.5%. The exact-match grader reproduces the strictness trade-off that [5] reports as an operational fact (a SymPy-backed replacement removes the measured false positives, but acceptance falls from 0.500 to 0.400 and coverage from 0.900 to 0.760): 0.0% false-accept at 24.4% false-reject, the rejected controls including the certificate's midpoint printed at full double precision and read back. The certificate grader breaks the trade-off, 0.0% and 0.0%, because it does not narrow the accepted language heuristically; it defines it as the set of values not yet refuted. The curve is the figure that answers the standard remedy: four orders of magnitude of tightening take absolute tolerance from 99.4% at 1×10^{-3} to 44.4% at 1×10^{-15}, a tolerance at the edge of what a double carries, and Proposition 1 says why the fall is slow: the band closes fact by fact as tol drops below half that fact's width, so the aggregate reaches zero only when the tolerance is under half the width of every certificate in the corpus, and at 1×10^{-15} it still accepts the 44.4% whose certificates are wider than 2×10^{-15}. The relative grader is worse at the tight end (69.8%) because a relative slack on a quantity of order 10^{-2} is a smaller absolute slack.

One canary that is not synthetic. The fact `erdos852.cstar`, $C^* = \frac{1}{2}\big(\prod_{p\geq 3}(1+(p-1)^{-3})-1\big)$, carries the value 0.0752403861777 that the Erdős #852 discussion thread published for it [21]. The certificate, as doubles, is [0.075240386178309235, 0.075240386178309582], of width 3.47×10^{-16}; the published value lies 6.09×10^{-13} outside it and inside any ordinary tolerance of it. It is a member of the acceptance band nobody had to mint: it was printed, in public, as a result, and the absolute grader accepted it 7 times of 9.

The gym and the attacker. The same record carries two model-facing environments on the corpus. The *uniformity gym* asks for a verdict with evidence on 12 instances (10 carrying a needle a sampling grid steps over), scored +1 correct with evidence, 0 correct but unsupported, +0.25 honest abstention, −1 wrong; evidence is a dyadic tiling accepted only if it forms a complete prefix-free set, checked combinatorially with no slack. A bare verdict scores zero however correct; a sampling grid dressed as a tiling scores −6.00, worse than abstaining, because the cell holding the needle refuses to verify; the sound interval solver scores +11.25 with one abstention, and the budget-limited sound solver, the row people omit, abstains 4 times and is never wrong (+9.00)—without it the +0.25 is never earned and the environment teaches bravado. The *attacker* inverts the polarity: shown a grader and a certificate, the model must produce a value the grader accepts and the certificate refutes, or NO_ATTACK. Of its 7 rungs, 3 have no attack (a tolerance narrower than the enclosure; a grader that compares against the certificate; a tolerance finer than the spacing of doubles around an integer, Proposition 2), and claiming a break that does not verify is scored wrong, not as a near miss. 34 forgeries are planted before any model is called and 0 have leaked; the battery (13 checks, 5 red controls) includes the two degenerate graders, accept-everything and reject-everything, which must both score zero.

5 When the answer key is wrong: three specimens and a control

An evaluation graded against reference values inherits the failure class of whatever computed them, and for mathematical ground truth the pipeline is usually floating point, checked by digit agreement. Three specimens from the repository's audits show the class; each is re-proved by the program that writes this paper's numbers. A fourth case, a published claim that holds, is the control, because an audit that only ever refutes is a search.

Specimen I: the constant that was a rounding error. The published C^* of §4 agrees with the certified lower endpoint 0.075240386178309245 5893 to 11 decimal places and is wrong from the 12th. The wrong digits are not near the truth by accident: re-run while this paper is built, the naive double-precision product over the 148,932 odd primes below 2,000,000 emits 0.0752403861777419, the published value digit for digit, because $1+(p-1)^{-3}$ rounds to exactly 1.0 once $(p-1)^3 \geq 2^{53}$, that is for $p-1 \geq 208{,}064$, so 87.5% of the factors (130,293) vanish and the product self-truncates regardless of the loop bound. The exact partial product is a strict lower bound of the full product with no tail estimate, and it already exceeds the published value under both truncation and rounding readings: REFUTED, in one integer inequality. The certificate itself is a directed-rounding product over 1,857,858 odd primes below 30,000,000 at 192 bits with the tail bounded analytically (22 decimal places). The thread's other constant c_0, the positive root of $I_0(c) = 1$, is VERIFIED as a rounding to 14 places (1.32322827686395) against a bracket of 200 bisections at 320 bits, with a wrong trailing ellipsis. In an answer key

the published C^* would score a float-faithful model correct and an exact one wrong, and a rerun of the same pipeline would confirm the key.

Specimen II: digit agreement is not evidence. Validating a key by matching digits assumes agreement implies identity. Over 14,677 OEIS decimal expansions [22], 21 published constants agree with simple closed forms and are provably not equal to them, each refutation one exact big-integer comparison at the full published precision. The deepest, re-derived here, is A271880 ("Decimal expansion of the probability that a random real number is evil."), which agrees with the rational $1/5$ for 62 significant digits before exact arithmetic separates them (the OEIS records the difference as A271881). A digit-matched validation shallower than that certifies the impostor; a double-precision check, at seventeen digits, cannot settle it at all.

Specimen III: true computation, false print. The key can be wrong although the computation behind it was right. The 7 result sheets of the Ramanujan Machine [20] print 51 rows; decided by rigorous enclosures and exact rational comparison, 50 survive and 1 is refuted: a mixed-zeta identity whose constant term carries a sign slip, false as printed, while the continued fraction its own polynomials define converges to the corrected value exactly, both directions re-certified on one enclosure. No rerun of the discovery pipeline catches this, because the pipeline was never wrong.

The control: a published claim that held. Pointed at the Lyapunov functions that arXiv:2606.10045 reports as discovered by constrained symbolic regression [17], an exact rational sum-of-squares certifier returns CONFIRMED for the function the paper states as valid, $V = x_1^2 + 4x_2^2$: $\dot{V} = -2x_1^2 - 8x_2^4$ is an exact weighted sum of squares, zero only at the origin. The three under-sampled candidates the paper flags itself are refuted with rational witnesses; for Eq. 23, $V = x_1^2 + 3.97x_2^2$, the point $x = (3/200000, 1/1000)$ gives $\dot{V} = 22103/50000000000000 > 0$. And run on the Motzkin polynomial, nonnegative everywhere and not a sum of squares [19], the same certifier returns REFUSED: it declines to certify a true statement for want of the certificate it requires, printing the probe $M(1,1) = 0$ beside the refusal. The 3 programs ran during this build and reported 19 passing rows including the red controls. A grader you have never seen decline is not known to be able to decline, and a grader that cannot decline will eventually confirm an answer key back to its author.

6 GSM8K's answer key, re-decided to the last step

Every harness that scores GSM8K [6] compares a model's number with the `####` answer line of a stored solution, and the solution prints the arithmetic that reaches it: calculator annotations `<<lhs=rhs>>` the authors inserted, and the prose steps between them. That arithmetic can be re-decided without a model and without a reader. GSM8K is held at commit 3101c7d of the authors' repository (1,319 test and 7,473 train items) and GSM8K-Platinum [7], the human revision of the test set, at revision e762492 of its dataset; both are re-hashed at every run. An annotation is read as the expression its left side denotes, by a hand-written parser for $+ - \times \div$ and parentheses over exact fractions with nothing passed to `eval`, and compared with the rational its right side denotes: EXACT, ROUNDED (within half a unit of the last printed place, and said so), WRONG or REFUSED. A prose equation is a run of arithmetic characters around an "=", read as a chain pair by pair with the notations the writers use (a dollar or percent sign

as a unit label, "5 and 2/3" as a mixed number, "25 / 1/3" as $25 \div \frac{1}{3}$); anything cut off by a word, glued to a variable or otherwise unreadable is UNREAD, never a verdict. The answer line is read as a rational and compared with the last annotation's value, then with every prose step's. Table 3 classes the items.

class	test	train
reproduced: every annotation exact; the answer is the last annotation's value	1,207	6,981
answer in prose: as above, the answer being a prose equation that checks exactly	50	200
answer underived: no arithmetic fault; the answer is no printed value	47	193
rounded step: a printed step is a rounding of its exact value	0	2
printed step wrong: a step is wrong by more than a rounding	2	24
no steps: no equation of either kind is printed	13	71
refused: an annotation outside the calculator grammar	0	2

Table 3: GSM8K items by whether their own printed steps reach their own answer, from `certs/gsm8k-ledger.json`. Test: 4,282 annotations, 3,572 prose equations read and 318 unread. Train: 23,716 annotations (23,714 exact, 2 refused for an operator outside the grammar), 19,671 read and 1,711 unread.

Findings. All 4,282 test-set annotations evaluate exactly to the value they print, not one wrong and not one rounded, and so do 23,714 of the 23,716 train-set ones (the other 2 use `//`). Reading the prose too, 2 test keys and 24 train keys print a step that does not hold as printed (Table 4); the answer rests on the wrong step in 1 test and 6 train keys and is held by another step of the same key in the rest, 18 of the train slips standing beside a correct calculator annotation. Two train steps are roundings (items 3390 ($10/3 = $3, exact 10/3); 4504 (13 / 6 = 2, exact 13/6)). Nothing here changes an answer a harness grades against: the answer of test item 501 is the annotation beside the slip, and item 1024's 300 is the number the writer meant by "$320 − $20". These are provenance findings about the key's text, the kind a few-shot prompt copies verbatim into a model's context, since both harnesses draw their examples from the train set.

Against Platinum. GSM8K-Platinum ran frontier models over the test set, inspected by hand the 219 items on which some model disagreed with the key, removed 110, relabelled 10 and verified 99, leaving 1,100 consensus items that no model disagreed with and nobody inspected. Joined by question text, exactly, the two readings cross-tabulate as in Table 5. Both test-set slips are consensus items: every model reproduced the intended answer, so a flagging method that looks only where a model disagrees had no reason to open them; a typo no model copies is invisible to a model-flagged audit by construction. The converse holds too: all 10 relabelled answers have arithmetic that holds to the last step (9 reproduced, one underived), and 103 of the 110 removed items are reproduced here. Those errors are readings of the problem, a rate read as linear or a quantity counted twice, and no re-decision of the numbers can see them. The two halves of a task-QA audit find different things, and the mechanical half runs in 1.4 seconds.

The graders. Every test answer is an integer as printed (1,303 plain, 14 with thousands separators and 2 negative); the train set has the same shape (7,391 plain, 79 with separators and 3 negative). Read against the two scoring rules pinned by commit (`inspect_evals` at

set	item	the printed step	exact value of the left side	answer	answer held by another step
test	501	364 / 4 = 273	91	273	yes
test	1024	$32 - $20 = $300	12	300	no
train	167	192*100 = $1920	19200	1920	no
train	168	6*2.5 = $18.00	15	54	yes
train	198	6/3 = .5	2	30	no
train	224	4080 + 4080 / 2 = 4000 + 2040	6120	20655	yes
train	502	3/1 = 1	3	6	yes
train	716	100 * 0.75 = $0.75	75	265	yes
train	3002	8*3 = 240	24	14	yes
train	3334	$4200+$3000 = $8600	7200	7200	no
train	3409	10 x .5 = 10	5	5	no
train	3859	5 * 3 = $15,000	15	90,000	yes
train	4593	14 / 1 / 4 = 14 * 4	7/2	4	no
train	4691	2*8 = 24	16	570	yes
train	4973	200*.55 = 100	110	1100	yes
train	5195	5 x 1/2 = 10	5/2	1	yes
train	5396	1 * 2 = 20	2	130	yes
train	5781	7+9+1+2 = $19.50	19	78	yes
train	6256	10 x 1,248 = 11,480	12480	27	yes
train	6590	1-2 = 96 - 70	-1	26	yes
train	6700	2 / 12 = .125	1/6	1	yes
train	6713	3 x (8/16) = 6	3/2	18	yes
train	6812	50-8 = 44	42	8	yes
train	6842	500/ 2/5 = 200	1250	250	yes
train	7178	1152/ 1,536 = .5	3/4	1	yes
train	7444	14*20 = 140	280	280	no

Table 4: Every printed step in GSM8K that does not hold as printed. Two train rows (5396, 6590) read a day number and a week range as arithmetic; they are listed rather than excused, because the rule that would exclude them is not one the reader can state.

Platinum status	items	reproduced	answer in prose	underived	no steps	step wrong
consensus	1,100	1,008	42	36	12	2
verified	99	87	4	7	1	0
removed	110	103	4	3	0	0
revised	10	9	0	1	0	0

Table 5: GSM8K-Platinum's statuses against this paper's classes, on the 1,209 kept and 110 removed test items.

788f5a2, whose `match(numeric=True)` parses the target as a number, observed on `inspect_ai` 0.3.266; `lm-evaluation-harness` at d6de816, which strips commas from both sides before an exact match), no key is in a form either harness misreads: of 8 probe forms observed, 6 grade correct and the two that do not (a fraction, a percent sign) occur in no answer line of either set. A false start is worth recording: the first survey listed annotations and answer lines in one list, so an annotation `<<3/4=3/4>>` was written up as a fraction-form key and an issue drafted on it; the report builder's gate (fraction keys = 0 $\neq$ the claimed 1) refused the page and the draft was deleted. The battery (51 checks, 12 red controls: a planted wrong annotation, a value off by more than half a unit, the `//` operator, a Python name, a power) ran during this build.

7 METR's time horizon, certified

The 50% time horizon of METR [8] is the length of task, measured in the time a human takes, that a model completes half the time; METR reads it off a logistic fit of run success on $\log_2$ human minutes and publishes a point estimate with a bootstrap interval, and the doubling time of that number is the most-quoted trend in AI forecasting. The evidence is public: the raw runs of the analysis repository [10] at commit 52cb829 (2026-03-06; 24,008 runs over 21 aliases) and the two files behind the live chart, fetched 2026-09-22 (the page says updated May 8, 2026): the per-task file with each agent's fitted coefficient and intercept to three decimals, and the benchmark file with the point estimates, bootstrap intervals, state-of-the-art flags and doubling times; the Time Horizon 1.1 post of 2026-01-29 [9] is pinned too, all seven files by sha256.

The estimator, to the letter. scikit-learn's weighted, L_2-penalised logistic regression is the minimiser of

$$J(w,b)=\sum_i s_i\big[-y_i\log p_i-(1-y_i)\log(1-p_i)\big]+\tfrac{\lambda}{2}w^2,\qquad p_i=\sigma(wx_i+b),\ x_i=\log_2(\text{minutes}_i),$$

with s_i the task weight ($1/\sqrt{k}$ for a family of k tasks, normalised to one) and $\lambda=10^{-5}$ from METR's parameter file. J is strictly convex, so the minimiser is the unique zero of the score $F(w,b)=\big(\lambda w-\sum_i s_i(y_i-p_i)x_i,\ -\sum_i s_i(y_i-p_i)\big)$, and per-run fitting collapses to per-task fitting with the mean outcome, exactly: $\sum_{\text{runs}}(s/n)(y_r-p)=s(\bar y-p)$. The horizon at success rate q is $2^{(\operatorname{logit} q-b)/w}$.

Proposition 3 (certification of the fit)**.** *Let X be a box in (w,b)-space, x_0 its centre, A a nonsingular matrix and $J_F(X)$ an interval enclosure of the Jacobian of F over X. If the Krawczyk image $K(X)=x_0-AF(x_0)+(I-AJ_F(X))(X-x_0)$, evaluated in outward-rounded interval arithmetic, lies in the interior of X, then F has exactly one zero in X [11, 13], and that zero is the minimiser of J.*

A float Newton iteration finds the candidate; the Krawczyk operator, evaluated in a standard-library interval arithmetic written for this (outward rounding by `nextafter`, exp and log from rational series with proved tails, every minute literal read as the rational it denotes and enclosed), proves the box, doubling its radius until containment holds or refusing; the horizon is the interval extension over the box. The weights are METR's, $1/\sqrt{k}$ as the correctly rounded double taken as a rational, which the file's own weights match to 2.9×10^{-9}.

What is proved, per model. All 44 fits certify: for each of the 23 agents on the live chart and each of the 21 aliases of the runs file, the box has radius below 10^{-10} (the largest is 3.6×10^{-13}). For Claude 3.7 Sonnet on the live file (228 tasks) the slope box is $[-0.5974549266701,-0.5974549266681]$, the intercept box $[3.5346505939786,3.5346505939859]$, and the 50% horizon $[60.387561330,60.387561332]$ minutes; METR prints 60.388937 with bootstrap interval $[33.0,104.2]$. Table 6 reads every printed number against its enclosure. The printed slope and intercept are the rounding of the box for 22 of 23 models; the exception is Claude Mythos Preview (early), whose intercept prints 5.582 where the box, 5.582691…, rounds to 5.583. The site's p50 estimates lie between 9.6×10^{-7} and 1.8×10^{-4} (relative) from the enclosure and never inside it (0 of 23), which is what a solver stopped at a tolerance looks like next to a proof; every enclosure lies inside METR's bootstrap interval. The March runs and

the May per-task file agree, where an agent is in both (20 aliases), to 2.2×10^{-5} on the horizon. The `human` alias of the runs file, fitted by the same rule on 188 tasks, gives 106.4 minutes; it is reported as a curiosity, not a measurement of people.

model	released	certified p50	site p50	gap	printed (w, b)	verdict
GPT-4 0314	2023-03-14	3.987	3.987	8.6×10^{-5}	$-0.641, 1.278$	reproduced
GPT-4 1106	2023-11-06	4.045	4.045	4.2×10^{-5}	$-0.585, 1.18$	reproduced
Claude 3 Opus	2024-03-04	3.952	3.952	3.1×10^{-6}	$-0.527, 1.046$	reproduced
GPT-4 Turbo	2024-04-09	3.733	3.733	2.3×10^{-5}	$-0.69, 1.312$	reproduced
GPT-4o	2024-05-13	6.991	6.991	8.4×10^{-5}	$-0.563, 1.578$	reproduced
Claude 3.5 Sonnet (Old)	2024-06-20	11.394	11.395	1.1×10^{-4}	$-0.501, 1.757$	reproduced
o1-preview	2024-09-12	20.329	20.327	1.4×10^{-4}	$-0.63, 2.737$	reproduced
Claude 3.5 Sonnet (New)	2024-10-22	20.523	20.523	6.0×10^{-6}	$-0.465, 2.026$	reproduced
o1	2024-12-05	38.831	38.832	2.1×10^{-5}	$-0.565, 2.983$	reproduced
Claude 3.7 Sonnet	2025-02-24	60.388	60.389	2.3×10^{-5}	$-0.597, 3.535$	reproduced
o3	2025-04-16	119.733	119.733	9.6×10^{-7}	$-0.694, 4.791$	reproduced
Claude 4 Opus	2025-05-22	100.372	100.366	6.3×10^{-5}	$-0.604, 4.014$	reproduced
Claude 4.1 Opus	2025-08-05	100.472	100.472	2.1×10^{-6}	$-0.661, 4.393$	reproduced
GPT-5	2025-08-07	202.996	203.013	8.3×10^{-5}	$-0.576, 4.417$	reproduced
Gemini 3 Pro	2025-11-18	224.344	224.326	8.3×10^{-5}	$-0.676, 5.279$	reproduced
GPT-5.1-Codex-Max	2025-11-19	223.727	223.715	5.4×10^{-5}	$-0.647, 5.048$	reproduced
Claude Opus 4.5	2025-11-24	293.003	292.995	2.9×10^{-5}	$-0.54, 4.425$	reproduced
GPT-5.2	2025-12-11	352.245	352.249	1.2×10^{-5}	$-0.574, 4.855$	reproduced
Claude Opus 4.6	2026-02-05	718.937	718.807	1.8×10^{-4}	$-0.412, 3.912$	reproduced
GPT-5.3-Codex	2026-02-05	349.496	349.531	1.0×10^{-4}	$-0.518, 4.379$	reproduced
Gemini 3.1 Pro	2026-02-19	384.189	384.147	1.1×10^{-4}	$-0.661, 5.676$	reproduced
GPT-5.4	2026-03-05	341.741	341.735	1.7×10^{-5}	$-0.52, 4.378$	reproduced
Claude Mythos Preview (early)	2026-04-07	1044.645	1044.780	1.3×10^{-4}	$-0.557, 5.582$	differs

Table 6: The 23 models of the live chart in release order: the certified 50% horizon (midpoint of the enclosure, in minutes; the enclosures are 2×10^{-11} to 4×10^{-8} minutes wide), METR's point estimate, their relative gap, and METR's printed slope and intercept against the rounding of the certified box. From `certs/horizon-ledger.json`.

The trend, as an interval. Over the models the site flags as state of the art at release with a horizon under 16 hours (the site's own rule, which leaves Claude Mythos Preview (early) out and Claude Opus 4.6 in), the exact least-squares line through $\log_2$ of the certified horizons against release date has a slope enclosed to 8.9×10^{-14} bits per day, a doubling time enclosed to 1.5×10^{-9} days: 128.740 days from 2023 over 14 models, where the site prints 128.744 with bootstrap interval $[104.428, 158.012]$ (Figure 2). From 2024 the same rule gives 104.7 days over 12 models; the January post printed 88.6 on its smaller set and 130.8 from 2023. So the site's trend is computed on exactly the numbers its chart shows by exactly the rule its file states. Where the January post and the May chart disagree, the certificate sides with the May file because it is the May file: of the 7 TH1.1 horizons the post printed, 1 (Claude 3.7 Sonnet, 60) is the rounding of the certified one and 6 differ by between 0.6% and 12% at the post's printed precision, the site's own May estimates having moved by the same amounts. Nothing was mis-computed: runs were added between the post and the chart, and the post is a fit on January runs that no public file holds. A printed number without the file it came from cannot be re-decided; it can only be dated.

Why the instrument exists. The page this comes from is a calibration. The same instrument is built to fit a horizon on tasks graded by an exact verifier (the environments of §8) against

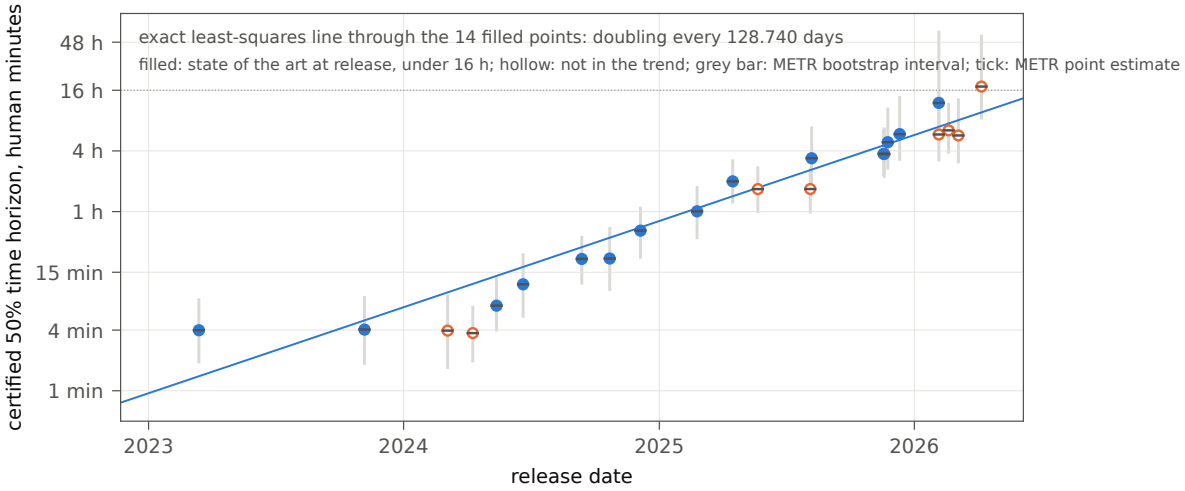


Figure 2: Certified 50% horizons against release date, from `certs/horizon-ledger.json`. The line is re-fitted by the drawing program from the 14 filled points and refused if its slope leaves the ledger's enclosure; the grey bars are METR's bootstrap intervals, not re-decided.

timed human baselines; today its own section reads NO DATA: 15 frontier logs with 180 rollouts on 18 listed tasks, none of which yet has a human time. The battery (33 checks, 7 red controls: a Taylor series without its remainder, a division through zero, Krawczyk from a far candidate, a slope box through zero asked for a horizon, a printed horizon moved by 1%) re-certifies one agent live from the pinned file and requires the ledger's box; the whole ledger re-derives in about 120 seconds on eight cores.

8 The verified reward channel and three environments

The channel. The design that removes the key is to grade tasks where no reference value exists to contaminate: ask the model to *exhibit* a witness and let the grader re-derive the claim from the witness alone, exactly. The running instance is a rank-R decomposition of the $\langle n, m, p\rangle$ matrix-multiplication tensor over $\mathbb{Q}$ or $\mathbb{F}_2$, the task on which proposer-plus-verifier produced a famous discovery [14, 16]: a proposal either satisfies all $(nm)(mp)(np)$ tensor equations in exact rational arithmetic or it does not, floats are refused at the door, and the verdict is CERTIFIED, REFUTED with the first violated equation and its exact discrepancy, or REFUSED. Three rules carry the discipline: a screen may only prune, never admit; every certifier is calibrated against a known answer and against red controls that must fire at import and at every build, among them a coefficient perturbed by exactly 10^{-9}, invisible to any float screen, a control that certifies aborting the campaign; and feedback to a proposing model is template-locked to the grader's own mechanism, so the channel can neither coach nor be sweet-talked. Across 12 campaigns of 3 frontier models, 364 ledger rows and 324 answered proposals (40 were cut off by the harness's own output cap and belong in no rate), 115 are certified, 147 rejected by the screen, 39 malformed, 23 declined and 0 refuted: every well-formed proposal that survived the float screen was exactly right, so frontier failures here are loud, not subtle. One row of the ledger is a correction: 90 rows tagged as run at low thinking effort ran at the default effort because the campaign path

dropped the flag (found 2026-08-31); the ledger is append-only, the correction file is read by every consumer, and the rows are displayed as default-effort measurements. The honesty probe asks for rank 6 on $\langle 2, 2, 2 \rangle$, impossible since the rank is 7 [15]: of 10 replies each, Claude Opus 5 declined 6, Claude Sonnet 5 5 and Claude Haiku 4.5 0, and no proposal certified. In closed loop, where the grader answers each failure with its own refutation and the model retries in the same conversation, 10 trajectories and 31 graded rounds ended with 6 certified, every one at its first round and 0 converted by feedback: a below-bar model is not rescued and an at-bar model needs no rescue.

break-the-grader. The attacker of §4, standard library only, is on the Environments Hub as `carlos-toledo/break-the-grader`. Its corpus is the 104 facts (103 pinned, 52 exact integers, stored as rationals). Of 2,000 generated tasks, 1,395 are breakable and 605 are not, decided by constructing an attack and never by a label; always-attack solves 1,175 and never-attack 645, so both standing answers lose and only checking wins. Difficulty is drawn in room (Proposition 2) from a declared mix of impossible 25%, razor 35%, narrow 25% and wide 15% and realised as 605, 453, 664 and 278 of 2,000, both figures printed because keys sit at five positions across the certificate and only the midpoint gives the closed form exactly. 1,366 forgeries are planted in the package's tests and 0 leaked; the build refuses the page if the careful policy's lead over blind play on the 400-task draw falls below a quarter of a point (it is 0.410). Table 7 is the reference table: on the same 120 seeds as the four policies (360 calls, $1.92 of a $4.00 cap reserved worst-case before every call, 2026-09-04), the three models separate, +0.950, +0.670 and +0.317, and the smallest scores below the one-line blind policy (+0.558), not because the tasks are hard for a program but because blind play makes no false claims and that model makes 23. The impossible column is the whole of what the environment measures; a mean that ignores it can be bought by guessing.

player	n	mean	solved	false claims	impossible	razor	narrow	wide
never (policy)	120	−0.350	32.5%	81	39/39	0/21	0/39	0/21
always (policy)	120	+0.558	55.8%	0	0/39	14/21	32/39	21/21
naive (policy)	120	+0.258	49.2%	28	0/39	11/21	28/39	20/21
careful (policy)	120	+1.000	100.0%	0	39/39	21/21	39/39	21/21
claude-opus-5 (effort low)	120	+0.950	96.7%	2	37/39	20/21	39/39	20/21
claude-sonnet-5 (effort low)	115	+0.670	80.9%	16	34/37	14/20	26/38	19/20
claude-haiku-4-5 (default)	120	+0.317	50.8%	23	25/39	5/21	17/39	14/21

Table 7: break-the-grader on 120 seeds, from `certs/grader-pilot.json` (scored by the shipped package). The policies read only the prompt: never (always refuse), always (attack at the first double past the interval), naive (submit hi + tol/2 in floats), careful (the arithmetic in exact rationals). Rung columns are solved/of; replies truncated by the caller's 2,000-token cap (5 for Sonnet) are excluded from the rates.

blind-spot. A kissing predicate for vectors in $\mathbb{Z}^{11}$ with 3-bit coordinates was put into RTL and proved sound; yosys then mutated the netlist 400 times, one bent bit of one cell's port each, and a testbench of pair verdicts drawn from finished certified mathematics could not see the comparator constant, because a corpus of valid inputs cannot test the code that rejects invalid ones, never varies the norms and never makes a large dot product. Four constructed families got to zero survivors; the interesting region is measure-zero to a sampler and reachable by hand, and

the environment asks whether a model can reach it: one mutation or none has been applied, and the model names up to eight input pairs that flip an output pin, claims EQUIVALENT, or says UNDECIDED. A kill is verified by simulating the netlist under the mutation, equivalence against a SAT proof on a miter, every label certified twice; the pool holds 348 killable, 51 proved equivalent and 1 identity mutant, 302 killed by the corpus and 8, 35 and 3 only by one constructed family each. Three rungs state more or less of the defect (*located*, *profile*, *blind*); scoring is +1 for a verified kill or a proved equivalence, 0 for a miss, an abstention or an unreadable reply, −1 for a kill claimed on a design proved unchanged or an equivalence claimed where a pair exists. Under Inspect [23] (0.3.266, seed 2027) the scorer is the package's own decision function; every log is re-scored offline, 180 frontier and 108 control rollouts with 0 disagreements, and the reference policies score +1.000 (the witness or the proof), 0.000 (abstain) and −0.556 (equivalent always) on 36 tasks. Table 8 is the effort ladder: Claude Sonnet 5 on the located rung goes 8, 10, 12 of 12 as effort rises, a budget ladder on a judge-free task; the blind rung carries 3 false alarms at every setting for every model, each a kill claimed on a design with no defect; and Claude Opus 5 declines the profile rung (10 of 12 unreadable, a content-policy refusal the prompt was not reworded to pass) while solving 10/12 located. The human-baselines file is pinned with 18 task ids and 0 attempts: NO DATA.

model, effort	located	profile	blind	wrong	unreadable	US$
Claude Haiku 4.5, default	3/12 (+0.08)	4/12 (+0.17)	4/12 (+0.08)	7	0	0.25
Claude Sonnet 5, low	8/12 (+0.58)	6/12 (+0.50)	3/12 (0.00)	4	1	0.30
Claude Sonnet 5, medium	10/12 (+0.83)	8/12 (+0.67)	5/12 (+0.17)	3	0	0.83
Claude Sonnet 5, high	12/12 (+1.00)	6/12 (+0.42)	5/12 (+0.17)	4	4	2.08
Claude Opus 5, low	10/12 (+0.83)	2/12 (+0.17)	4/12 (+0.08)	3	14	0.40

Table 8: blind-spot under Inspect, 12 tasks per rung: solved/of (mean reward), from `certs/blind-spot-inspect-ledger.json`. Unreadable replies are refusals or over-budget replies, scored 0 and never masked.

lattice-claims. The third environment is built from a mistake: auditing published SVP-challenge records, a bit-exact grader of the shortest-vector claim reported most of them as disagreeing with their published figure, every discrepancy at about 10^{-4} and in both signs, because the published ratios had been computed from the true norm and printed rounded while the exact ratio was computed from the rounded norm. Exactness did not save the grader; naming what the claim was about would have. So a submission must declare the reference it decided against, and a right verdict from a reference the task did not state does not score as right. Instances are minted from a seed around a short vector chosen first, the predicate is exact with a certified π bracket, and three rungs state more or less of the reference: *declared*; *printed*, where STRADDLES is a genuine third answer (at $n = 24$ a printed norm of 1,390 against a wall of 1,390 leaves the ratio in $[1.0493, 1.0501]$ across the threshold); and *underspecified*, where only NEEDS_DATA, naming the field, is right. 10 forgeries are planted and all caught, two of them holes in the laboratory's own grader. The float-versus-exact story was measured and is weaker than it looks: below dimension 102 a careful float grader agrees with the exact decision on every sampled instance, and above it, where the challenge-scaled determinant overflows a double, a naive float grader accepts everything (12 of 12 at dimensions 104–140 against 0 of 12 at 24–90); the failure in this domain is a confidently blind grader, not rounding. Through the framework (72 rollouts, 2026-09-10, $1.85, re-scored offline with no disagreement), Claude Sonnet 5 certifies

10, 2 and 4 of 12 on the three rungs and Claude Haiku 4.5 answers 4/12 of the declared rung correctly while declaring a usable reference on 0/12: every verdict it got right came from a reference it never properly stated, which is the environment's thesis appearing as a measurement.

9 Certified MathBench v0, pre-registered

Seven construction families ask for an exhibited object and decide it exactly, with no key, judge or tolerance: a Golomb ruler with n marks and length at most L; a cap of size k in $\mathbb{F}_3^n$; a binary code (n, M, d); a graph on n vertices with no K_s and no independent t-set; a law on $\mathbb{Z}^2$ with $H(X-Y) \geq c\max(H(X), H(Y), H(X+Y))$, entropies in intervals; k equal-norm integer vectors pairwise at least 60° apart; and a rank-R bilinear algorithm for n-term polynomial products over $\mathbb{F}_2$. The ladders run to the published optima and records, and 5 rungs ask beyond the record. The pre-registration of 29 September 2026, written before the first model call, fixed the families and ladders; one sample per rung at the API's default thinking and effort with a 24,000-token output cap; the three models (Claude Haiku 4.5, Claude Sonnet 5, Claude Opus 5; the keys on this machine reach no other laboratory); the metrics (certified of graded; refuted apart from malformed and budget-exhausted; rungs certified that the baseline does not; tokens and dollars per certified rung); a US$30 ceiling reserved worst-case before every call; and the null: if no model certifies a rung the baseline does not, the easy rungs are recall and the hard ones out of reach at one sample, and a certified beyond-the-record rung is not announced from this run but re-decided by a second implementation first. Every rung with a witness on record carries a green control that must certify and near-misses forged from it that must refute before any model is asked (28 green, 20 red; the battery of 83 checks ran during this build), and each family's textbook first try (a greedy ruler, a lexicode, a quadratic-residue circulant, the D_n roots, schoolbook multiplication) is decided like any proposal: the baseline certifies 6 of 46.

model	graded	cert.	ref.	rej.	malf.	capped	beyond	US$	$/cert.
Claude Haiku 4.5	46	5	35	5	1	0	2	0.21	0.042
Claude Sonnet 5	21	18	2	0	1	25	13	6.79	0.377
Claude Opus 5	23	21	1	0	1	23	15	15.31	0.729

Table 9: Certified MathBench v0 by model, from `certs/mathbench-ledger.jsonl` (184 rows; 46 rungs each): rungs graded, certified, refuted, rejected by the screen, malformed, capped (the reply reached the output cap before an object: recorded and billed, never graded), certified beyond the baseline, dollars spent and dollars per certified rung.

family	rungs	baseline	Haiku 4.5	Sonnet 5 (cap)	Opus 5 (cap)
Golomb rulers	6	0	2	5 (1)	6 (0)
cap sets in $\mathbb{F}_3^n$	6	0	0	1 (5)	2 (4)
binary codes	6	1	1	1 (4)	1 (5)
Ramsey graphs	7	2	1	2 (3)	3 (4)
sum–difference laws	8	0	0	0 (8)	0 (7)
kissing configurations	7	3	1	6 (1)	6 (0)
polynomial products over $\mathbb{F}_2$	6	0	0	3 (3)	3 (3)

Table 10: Rungs certified per family and answerer; in parentheses, the rungs on which the reply reached the output cap.

Results. Tables 9 and 10. Claude Haiku 4.5 certifies 5 of 46 graded rungs (2 the baseline does not reach) and is refuted on 35; Claude Sonnet 5 certifies 18 of 21 (13 beyond the baseline) and ran out of tokens on 25; Claude Opus 5 certifies 21 of 23 (15 beyond) and ran out on 23. The run cost US$22.30 of the ceiling. No model certified a rung beyond the published record (0 of 5); the null of the pre-registration is the result. The refutations of the two larger models are few and named: Opus 5's one refuted object, at kissing (5, 41), is 40 vectors, norms [2], span dimension 5, a pair too close [[1, 1, 0, 0, 0], [1, 1, 0, 0, 0]], a duplicate vector, exactly the near-miss the red controls forge; Sonnet 5's two are at ramsey (3, 4, 8) and ramsey (3, 6, 17), each with a triangle the certifier names. The finding of v0 is about the cap rather than about correctness: the two larger models reached 24,000 output tokens before emitting an object on more than half of the rungs, almost always where the objects are large (caps, codes, the entropy laws, the larger products), so v0 measured finishing, and a v1 with a larger cap or an explicit effort setting is to be pre-registered first. One deviation is recorded in the pre-registration: five Opus 5 calls on the sum–difference family returned an API error with no reply and nothing billed while the account's credit ran out, and were issued once more after the run with the same settings; no rung was sampled twice.

10 What is not claimed

The band proposition is elementary; the measurement of §4 is of four reference grader shapes, not of anybody's product, and the suite that measures a real grader runs on its owner's machine. One laboratory supplies every certificate in the corpus; that is the moat and equally the limit. Nothing here is a formal proof in the sense of a proof assistant; the standard met is the working one of computer-assisted proof, one rung below a kernel and several above a decimal that looked convincing. In §5 no priority is claimed for any mathematics: the Lyapunov functions are the paper's, the SOS-Lyapunov technique is Parrilo's [18], and arXiv:2606.10045 is cited as a claim whose statements are taken as printed. In §6 no answer is called wrong and no question ambiguous; the prose reader is conservative by design (318 test and 1,711 train equations unread and counted, never guessed), and the harness observations are of two pinned commits on one date. In §7 no horizon is called right or wrong: the certificate is about the fit, not the tasks, the human timings or the binarisation of scores, all METR's and taken as given; the bootstrap intervals are not re-decided, since a random draw is not a theorem and the certified box is the optimiser's uncertainty; the post's runs are not public. In §8 the matmul oracle decides finitely many exact-arithmetic facts and refuses everything else; blind-spot is one predicate, one netlist and one mutation operator; lattice-claims certifies arithmetic about a stated lattice and threshold and nothing about attack cost or any deployed scheme. In §9 one sample per rung is a Pass@1 at default settings, recall and discovery look alike since most rungs are known optima, and only one laboratory's models were asked. Two figures the environments' own documentation states, the size of the certified testbench corpus behind blind-spot and the record count of the audit behind lattice-claims, have no record in this repository and are deliberately not quoted.

Reproduction

Every constant in this paper is interpolated from a certificate record in the `cert-machine` repository, so the document cannot quote a number the machine did not certify. The records, and the commands that rewrite them:

- `certs/envs-record.json`, written by `node tools/run-envs.js`, offline;
- `certs/gym-record.json`, by `node tools/run-gym-record.js`; `certs/grader-pilot.json`;
- `certs/erdos852-certificate.json`;
- `certs/matmul-eval-ledger.jsonl`, its corrections file, and `certs/matmul-loop-ledger.jsonl`;
- `certs/gsm8k-ledger.json`, by `python3 tools/run-gsm8k-ledger.py`;
- `certs/horizon-ledger.json`, by `python3 tools/run-horizon-ledger.py`;
- `environments/blind_spot/pool/pool.json`; `certs/blind-spot-inspect-ledger.json`;
- `instruments/wiring/eval/`;
- `certs/mathbench-ledger.jsonl`, by `python3 tools/run-mathbench.py`, which reserves its ceiling before every call.

The macros of this document are written by `tools/paper-numbers/certified-evaluation.js` (run with `node`), which also re-runs the naive product and exact refutation of specimen I, the impostor depth of specimen II in exact big integers, both directions of specimen III, the three SOS programs, and the envs, gsm8k and mathbench batteries; the figures by `tools/paper-numbers/certified-evaluation-figs.py`. The environments are published as `carlos-toledo/break-the-grader`, `carlos-toledo/blind-spot` and `carlos-toledo/lattice-claims` on the Environments Hub [24]. Sources are pinned by sha256 in `corpus/metr-horizon/meta.json` and `corpus/gsm8k/meta.json`; this document was built at commit 16849460e608.

References

[1] Rajan, Auditing reward hackability in code RL, arXiv:2606.16062 (2026).

[2] OpenAI, note on test quality in SWE-bench Verified, February 2026.

[3] Zhang, When the reward suite is leaky, arXiv:2607.11022 (2026).

[4] LLMs gaming verifiers, arXiv:2604.15149 (2026).

[5] Ray, Fuzzing RLVR verifiers, arXiv:2606.01066 (2026).

[6] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, J. Schulman, Training verifiers to solve math word problems, arXiv:2110.14168 (2021); `github.com/openai/grade-school-math` at 3101c7d.

[7] J. Vendrow, E. Vendrow, S. Beery, A. Mądry, Do large language model benchmarks test reliability?, arXiv:2502.03461 (2025); `madrylab/gsm8k-platinum` at e762492.

[8] T. Kwa, B. West, et al., Measuring AI ability to complete long tasks, arXiv:2503.14499 (2025).

[9] METR, Time Horizon 1.1, `https://metr.org/blog/2026-1-29-time-horizon-1-1/` (2026-01-29); the live chart and its data files at `https://metr.org/time-horizons`, fetched 2026-09-22.

[10] METR, `eval-analysis-public`, `https://github.com/METR/eval-analysis-public`, commit 52cb829 (2026-03-06).

[11] R. Krawczyk, Newton-Algorithmen zur Bestimmung von Nullstellen mit Fehlerschranken, Computing 4 (1969), 187–201.

[12] R. E. Moore, R. B. Kearfott, M. J. Cloud, Introduction to Interval Analysis, SIAM, 2009.

[13] S. M. Rump, Verification methods: rigorous results using floating-point arithmetic, Acta Numerica 19 (2010), 287–449.

[14] V. Strassen, Gaussian elimination is not optimal, Numer. Math. 13 (1969), 354–356.

[15] S. Winograd, On multiplication of 2×2 matrices, Linear Algebra Appl. 4 (1971), 381–388.

[16] A. Fawzi et al., Discovering faster matrix multiplication algorithms with reinforcement learning, Nature 610 (2022), 47–53.

[17] Lyapunov functions discovered by constrained symbolic regression, arXiv:2606.10045 (2026); cited as a claim, its equations taken as printed.

[18] P. A. Parrilo, Structured semidefinite programs and semialgebraic geometry methods in robustness and optimization, PhD thesis, Caltech, 2000.

[19] T. S. Motzkin, The arithmetic-geometric inequality, in: Inequalities (O. Shisha, ed.), Academic Press, 1967, 205–224.

[20] G. Raayoni, S. Gottlieb, Y. Manor, G. Pisha, Y. Harris, U. Mendlovic, D. Haviv, Y. Hadad, I. Kaminer, Generating conjectures on fundamental constants with the Ramanujan Machine, Nature 590 (2021), 67–73; the result sheets at `https://www.ramanujanmachine.com`.

[21] Erdős problem #852 and its discussion thread, `https://www.erdosproblems.com/852`, fetched 2026-08-25 and pinned by sha256 in `certs/erdos852-certificate.json`.

[22] OEIS Foundation, The On-Line Encyclopedia of Integer Sequences, entries A271880 and A271881, `https://oeis.org`.

[23] UK AI Security Institute, Inspect, `https://inspect.aisi.org.uk`; `inspect_evals` at `https://github.com/UKGovernmentBEIS/inspect_evals`, commit 788f5a2; EleutherAI, lm-evaluation-harness, `https://github.com/EleutherAI/lm-evaluation-harness`, commit d6de816.

[24] Prime Intellect, the `verifiers` library and the Environments Hub, `https://github.com/PrimeIntellect-ai/verifiers`; YosysHQ, Yosys and `mcy`, `https://github.com/YosysHQ`.

[25] C. Toledo, cert-machine: independent exact certification of machine-generated mathematics, `https://github.com/carlostoledo1891/cert-machine`, doi:10.5281/zenodo.22225860; reports at `https://carlostoledo.co/reports/`.